RAG-Based Customer Support Assistant using LangGraph + HITL

---

1. Introduction

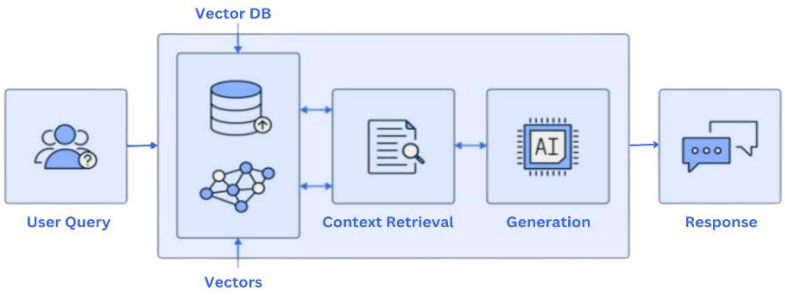
1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an advanced AI technique used to improve the quality and accuracy of chatbot answers. In a normal chatbot, the model gives responses only from the data it learned during training. Because of this, it may not know the latest company policies, internal documents, or business-specific information. RAG solves this problem by connecting the AI model with external knowledge sources such as PDFs, manuals, FAQs, databases, or policy documents.

The term RAG contains two important parts: **Retrieval** and **Generation**. Retrieval means searching and collecting relevant information from stored documents based on the user question. Generation means using that retrieved information to create a human-like final answer. This combination helps the chatbot answer based on facts rather than guesses.

For example, if a customer asks "What is the refund policy?", the system first searches the refund policy document, finds the correct section, and then gives that content to the LLM. The model then creates a clean and natural answer such as "Customers can request a refund within 7 days after delivery." This makes the system more reliable and useful.

RAG is widely used in modern AI products because it allows companies to use their own private knowledge with language models. It is one of the best solutions for enterprise support systems, healthcare assistants, legal document bots, and customer service platforms.



1.2 Why RAG is Needed

Traditional chatbots have many limitations. Most chatbots depend only on pre-trained model knowledge, which may be outdated or too general. They often fail when users ask company-specific questions such as return rules, billing procedures, or product warranty details. In such cases, the chatbot may guess an answer, which can reduce customer trust.

Another major issue is hallucination. Hallucination means the AI confidently gives wrong or imaginary information. For example, if the company refund period is 7 days but the chatbot says 30 days, it can create customer complaints and business loss. In real customer support systems, wrong answers can be risky.

RAG is needed because it solves these problems in a practical way. Instead of depending only on training knowledge, the system retrieves information from actual business documents. This means the answer is based on company-approved content. If the company updates a policy, only the document needs to be updated. No full model retraining is required.

Because of these reasons, RAG is highly useful in industries like banking, healthcare, e-commerce, telecom, education, and software companies. It increases accuracy, reduces manual workload, and improves user satisfaction.

1.3 Use Case Overview

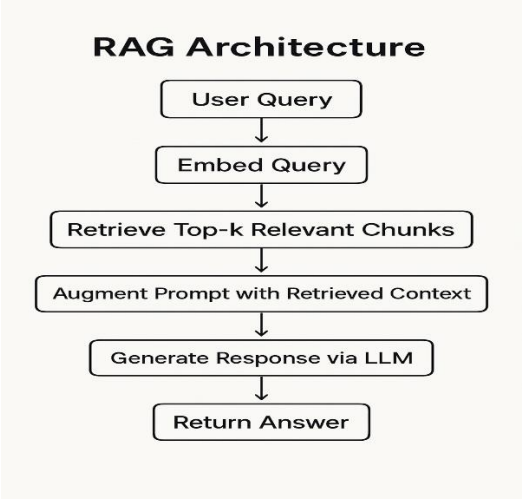
This project is designed as a **Customer Support Assistant** for businesses that receive repeated support queries every day. Many customers ask similar questions related to returns, shipping, subscriptions, billing, passwords, and product usage. Handling these repetitive tasks manually requires many support agents and increases cost.

The system stores business knowledge in PDF files such as return policy, FAQ manuals, shipping terms, billing rules, and warranty guides. When a customer asks a question, the assistant searches these documents and gives the correct answer immediately. This saves time for both customers and company support teams.

For example, if a user asks “How long does delivery take?”, the assistant retrieves shipping information and replies with the estimated delivery time. If a user asks “How can I cancel my order?”, the bot can explain the cancellation steps from the policy document.

If the question is complex, emotional, or unclear, the system can transfer the issue to a human support executive using Human-in-the-Loop escalation. This creates a balanced system where AI handles common tasks and humans solve difficult cases.

2. System Architecture Explanation



2.1 High Level Design (HLD)

High Level Design explains the complete structure of the system and how all components work together. It gives an overview instead of code-level details. In this project, the architecture includes the user interface, document processing module, embedding model, vector database,

retrieval system, LLM response engine, LangGraph workflow controller, and HITL escalation module.

When the system starts, company PDF files are uploaded and processed. Their content is converted into chunks and embeddings, then stored in the vector database. Later, when a customer sends a query, the system searches for relevant chunks and sends them to the LLM for response generation.

The workflow is managed by LangGraph, which helps route the query to either the AI response node or human escalation node. This makes the architecture intelligent and modular. Each part has a clear responsibility, which improves maintainability.

A well-designed HLD is important because it shows scalability, future expansion, system dependencies, and communication between modules. It helps engineers understand the project quickly.

2.2 User Interface Layer

The user interface is the point where customers interact with the system. It can be a website chatbot, mobile app chat window, internal support portal, or command-line interface during development. The UI collects the user question and displays the final answer.

A good UI should be simple, responsive, and user-friendly. Customers should be able to ask questions naturally in normal language. The interface may also provide buttons like “Talk to Human Agent” or “Rate This Answer.”

The UI sends the user query to the backend workflow system. After processing, the answer is shown in the chat window. If escalation is required, the UI should inform the user politely that a human support member will assist soon.

In production systems, the UI can also support multiple languages, file uploads, voice input, and session history for better user experience.

2.3 Document Ingestion Pipeline

The document ingestion pipeline is responsible for preparing company documents before they can be used in RAG. Since PDFs cannot be directly searched efficiently, they need preprocessing.

First, the system loads the PDF using tools like PyPDFLoader. Then the text is extracted from each page. Unnecessary spaces, symbols, and formatting issues are cleaned to improve quality.

Next, the long text is divided into smaller chunks. These chunks are converted into embeddings and stored in the vector database. Each chunk represents a meaningful section of the document, such as refund rules or warranty terms.

This pipeline is important because poor document preparation leads to poor retrieval results. Clean, structured, and properly chunked documents improve system accuracy significantly.

2.4 Embedding Layer

Embeddings are numerical vector representations of text. Instead of comparing exact words, embeddings help the system compare meaning. This is useful because users may ask the same question in different ways.

For example, “refund policy” and “how can I return my money?” use different words but have similar meaning. Embeddings place similar meanings closer in vector space, allowing semantic search.

This project uses sentence transformer models such as all-MiniLM-L6-v2. These models are lightweight, fast, and suitable for production systems. They convert both document chunks and user queries into vectors.

The embedding layer is one of the most critical parts of RAG. If embeddings are poor, retrieval quality drops. Good embeddings directly improve answer relevance.

2.5 Vector Database

ChromaDB is used to store document chunk embeddings. A vector database is different from traditional databases because it is designed for similarity search instead of exact keyword search.

When a user asks a question, the query is converted into an embedding. ChromaDB compares this query vector with stored chunk vectors and returns the most similar chunks.

This method is faster and smarter than normal keyword matching because it understands meaning. Even if exact words are different, relevant content can still be found.

ChromaDB is chosen because it is lightweight, easy to use, open-source, and integrates well with Python and LangChain ecosystems.

2.6 Retrieval Layer

The retrieval layer takes the user query and searches for relevant document chunks. It uses similarity scoring between the query embedding and stored embeddings.

Usually, the system returns Top-K results, such as top 3 most relevant chunks. Returning too many chunks increases cost and confusion, while too few chunks may miss important context.

For example, if the user asks about cancellation, the retrieval layer may return cancellation rules, refund conditions, and cancellation timing policy.

This layer is important because the final answer quality depends heavily on retrieved context. Good retrieval leads to good generation.

2.7 LLM Layer

The Large Language Model (LLM) is responsible for generating final natural language responses. It receives the retrieved document chunks and the user question as input.

Instead of generating freely, the model is instructed to answer only from the provided context. This prevents hallucination and improves trustworthiness.

Popular choices include OpenAI GPT models or open-source models like Llama and Mistral.

The LLM converts technical document text into clear customer-friendly answers, making the system natural and easy to use.

2.8 Workflow Layer (LangGraph)

LangGraph controls the system logic. Unlike a simple chatbot chain, LangGraph supports nodes, branching, conditions, retries, and state management.

Each step such as retrieval, generation, confidence checking, and escalation is represented as a node. Based on output, the graph decides next action.

For example, if confidence is high, go to answer node. If low, go to escalation node. This makes the system intelligent and scalable.

LangGraph is useful because real business systems need decision-making flows, not just one-step responses.

2.9 HITL Module

Human-in-the-Loop (HITL) means human experts join the process when AI is uncertain. This is essential in customer support where wrong answers can damage trust.

If the system cannot find relevant context or detects a sensitive complaint, it creates a support ticket. The human agent receives the user query and retrieved information.

The human then provides the final answer. This hybrid model combines AI speed with human judgment.

HITL increases reliability and customer satisfaction, especially for complex issues like refunds, disputes, or emotional complaints.

3. Design Decisions

Design decisions are important because they decide how well the system performs in real-world usage. In a RAG-based customer support assistant, every technical choice such as chunk size, embedding model, retrieval method, and prompt design affects speed, accuracy, cost, and user satisfaction. Good design decisions help create a stable and scalable system.

Instead of selecting settings randomly, this project uses balanced values that work well for customer support data like FAQs, policies, and manuals. These choices are made by considering performance, response time, and ease of future improvement.

3.1 Chunk Size Choice

Chunk size means how much text from the document is stored in one segment before converting it into embeddings. In this project, a chunk size of around **500 tokens** with **100 token overlap** is used. This gives enough context for the model to understand the topic while keeping chunks small enough for accurate retrieval.

If chunks are too small, important sentences may be split into separate parts, which can reduce answer quality. If chunks are too large, unrelated topics may mix together, making retrieval less accurate. Therefore, a medium chunk size is the best practical solution.

The overlap between chunks helps maintain continuity. If one paragraph ends near the boundary of a chunk, some part of it appears in the next chunk also. This improves context understanding and retrieval consistency.

3.2 Embedding Strategy

Embeddings are numerical vector representations of text used for semantic search. In this project, sentence transformer models such as **all-MiniLM-L6-v2** are selected because they are lightweight, fast, and accurate for text similarity tasks.

Embeddings help the system understand meaning instead of only exact words. For example, “refund policy” and “how can I get my money back?” have different wording but similar meaning. Good embeddings allow both questions to retrieve the same relevant content.

A lightweight embedding model is preferred because customer support systems need quick responses. Faster embeddings reduce waiting time and infrastructure cost while still giving strong retrieval performance.

3.3 Retrieval Approach

The retrieval layer searches document chunks and returns the most relevant ones. This project uses **Top-K semantic retrieval**, where the system returns the top 3 matching chunks based on similarity score.

Using only one chunk may miss important supporting details, while returning too many chunks increases token cost and adds noise to the prompt. Top-K = 3 gives a good balance between answer quality and efficiency.

For example, if the customer asks about order cancellation, the system may retrieve cancellation policy, refund after cancellation, and cancellation deadline. This gives enough context for a complete answer.

3.4 Prompt Design Logic

Prompt design controls how the LLM responds. In this project, the model is clearly instructed to answer only from retrieved context and avoid making assumptions. This reduces hallucination and increases trust.

Example prompt:

You are a customer support assistant.

Use only the provided context.

If answer is not available, escalate to human support.

Give clear and polite responses.

This prompt ensures answers remain professional, concise, and based on company documents. Good prompt design is essential because even a powerful model can give poor answers without proper instructions.

4. Workflow Explanation

Workflow means the sequence of steps followed when a user asks a question. Instead of directly sending the query to an LLM, the system uses a structured pipeline with retrieval, answer generation, decision making, and escalation.

This project uses LangGraph because customer support systems need branching logic and multiple possible outcomes. Some questions can be answered automatically, while others need human help.

4.1 LangGraph Usage

LangGraph is used to build the system as connected nodes. Each node performs a separate task such as retrieving chunks, generating responses, or checking confidence. This makes the architecture modular and easy to maintain.

Unlike simple chains, LangGraph supports conditional routing. If the answer confidence is high, it routes to the response node. If confidence is low, it routes to the escalation node. This makes the system smarter and production-ready.

LangGraph also supports state management, meaning data such as query, retrieved documents, and answer can move across nodes efficiently.

4.2 Node Responsibilities

Each node has a clear responsibility. The **Input Node** receives the user query. The **Retrieval Node** searches the vector database and returns relevant chunks. The **Generation Node** sends the query and chunks to the LLM to create a response.

The **Decision Node** checks confidence score, query intent, and whether enough context was found. Based on this, it decides whether to continue with AI answer or escalate to human support.

Finally, the **Response Node** returns the answer to the user, while the **Escalation Node** creates a support ticket for manual handling.

4.3 State Transitions

State means the shared data passed from one node to another. It stores important information such as user query, retrieved chunks, answer text, confidence score, and final route decision.

Example:

```
state = {
 "query": "",
 "docs": [],
 "answer": "",
 "confidence": 0.0,
```

"route": ""

}

Each node updates the state. This structured flow makes debugging easier and supports future upgrades such as memory, analytics, or conversation history.

5. Conditional Logic

Conditional logic is the decision-making part of the system. It determines whether the chatbot should answer automatically or send the issue to a human agent. This is important because AI should not answer every question blindly.

Without conditional logic, the system may give wrong answers on sensitive topics. By using rules and confidence checks, the assistant becomes safer and more reliable.

5.1 Intent Detection

Intent detection identifies what the customer wants. The system can classify questions into categories such as refund request, shipping query, billing issue, cancellation request, complaint, or technical problem.

For example, “How long does delivery take?” is a shipping intent, while “My refund is delayed” is a refund complaint. Detecting intent helps route the request correctly.

Intent detection also helps prioritize urgent cases. Complaints or payment issues may need faster escalation than normal FAQ questions.

5.2 Routing Decisions

Routing decisions are based on confidence score and detected intent. If confidence is high and relevant chunks are found, the system gives an AI-generated answer.

If confidence is low, no relevant document is found, or the query is sensitive, the system routes the issue to human support. This prevents automation mistakes.

For example, a question like “What is your warranty period?” can be answered automatically, but “Money deducted twice from my account” should be escalated.

6. HITL Implementation

Human-in-the-Loop (HITL) means human experts are included when AI cannot safely or accurately solve the issue. This creates a balanced support system where AI handles simple cases and humans solve complex ones.

HITL is very useful in production customer service because many real issues involve emotions, exceptions, disputes, or private account access.

6.1 Role of Human Intervention

Human agents handle situations such as refund disputes, failed payments, angry customer complaints, legal issues, or cases where no matching policy exists in documents.

When escalation happens, the system sends the customer query and retrieved context to the agent. This helps the human understand the issue faster and respond efficiently.

Human intervention adds empathy, judgment, and flexibility that AI cannot fully provide.

6.2 Benefits and Limitations

The main benefits of HITL are higher trust, safer responses, better customer satisfaction, and correct handling of edge cases. It also reduces the risk of AI making serious mistakes.

However, HITL also has limitations. Human support costs more than automation, and responses may take longer than instant AI replies. During peak hours, ticket queues may form.

Even with these limitations, HITL is necessary for high-quality enterprise support systems.

7. Challenges & Trade-offs

Every engineering system has trade-offs. Improving one area often affects another area such as speed, cost, or accuracy. Good design means choosing balanced settings.

This project carefully balances retrieval quality, response speed, context size, and operating cost.

7.1 Retrieval Accuracy vs Speed

Retrieving more chunks gives the LLM more context, which can improve answer quality. However, searching and processing more chunks increases latency.

Retrieving fewer chunks makes the system faster but may miss useful information. Therefore, Top-K = 3 is chosen as a balanced setting.

This provides good performance while keeping responses fast for customer support usage.

7.2 Chunk Size vs Context Quality

Small chunks are more precise because they focus on one topic. But they may lose paragraph-level meaning. Large chunks preserve more context but may contain unrelated text.

A medium chunk size of around 500 tokens solves this problem by keeping enough detail while maintaining retrieval quality.

This is why chunk tuning is an important part of RAG optimization.

7.3 Cost vs Performance

Using premium LLM APIs gives high-quality answers but increases operating cost. Open-source local models reduce cost but may give lower performance depending on hardware and tuning.

For startups or small companies, local deployment may be attractive. For enterprise systems requiring top quality, managed APIs may be better.

This project architecture supports either option based on budget and scale.

8. Testing Strategy

Testing ensures the system works correctly before deployment. It validates whether answers are accurate, retrieval is relevant, and escalation logic works properly.

A strong testing strategy is important because customer support systems directly affect users.

8.1 Testing Approach

Multiple testing types are used such as functional testing, retrieval testing, performance testing, and escalation testing. Each module is checked individually and then as a complete pipeline.

Functional testing checks whether common questions receive correct answers. Retrieval testing checks whether correct chunks are returned from the database.

Performance testing measures response time, system load, and stability under multiple users.

8.2 Sample Queries

Example test queries:

- What is refund policy?
- How many days for delivery?
- Can I cancel my order?
- How do I claim warranty?
- My refund is delayed.

Expected result is either a correct answer or escalation when necessary.

Using realistic sample queries improves system quality before launch.

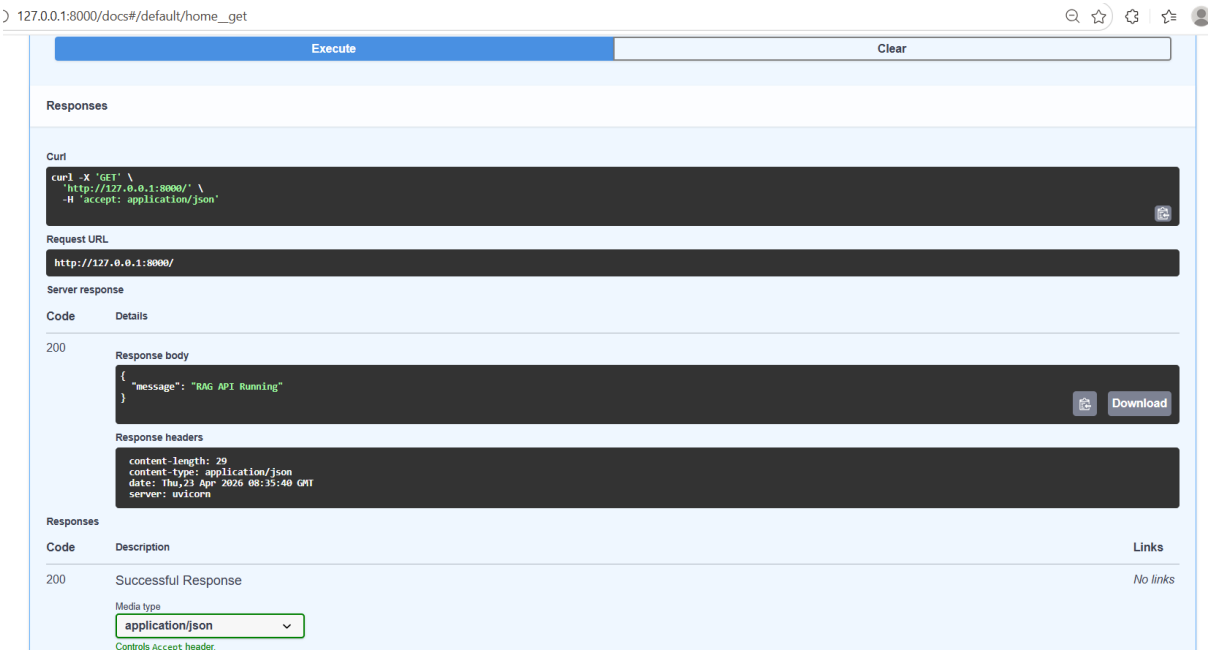
9. Future Enhancements

Future enhancements can make the assistant more intelligent, scalable, and useful for larger organizations.

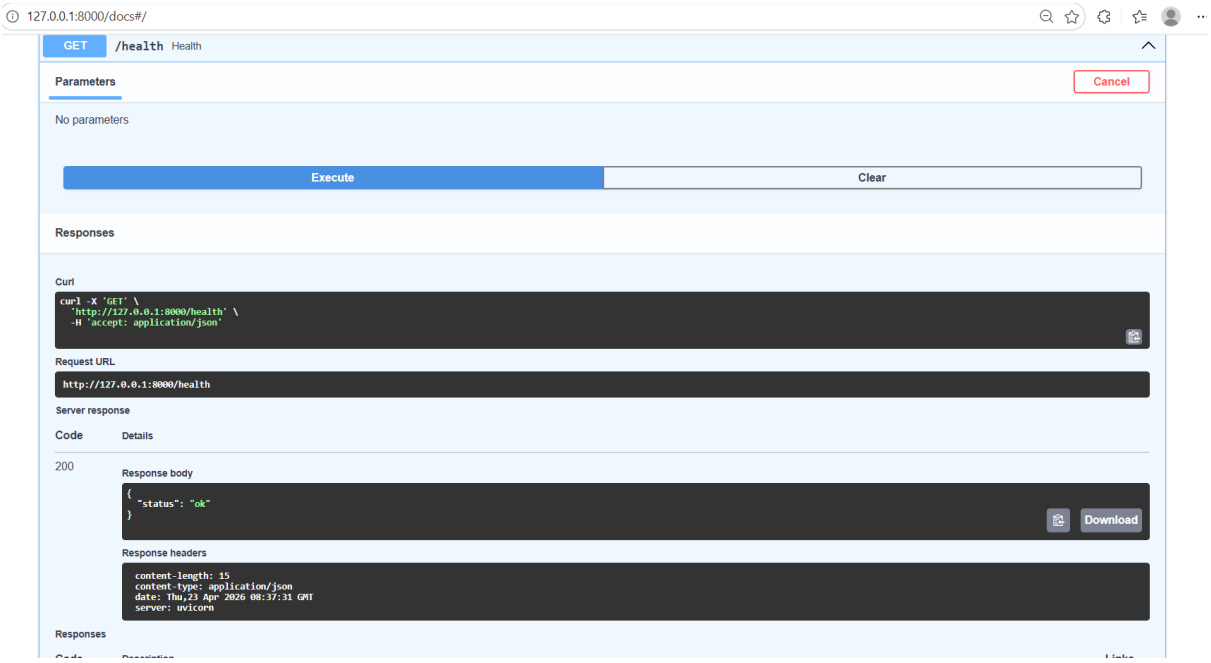
The current version focuses on core RAG + workflow + escalation features, but many advanced features can be added later.

Output

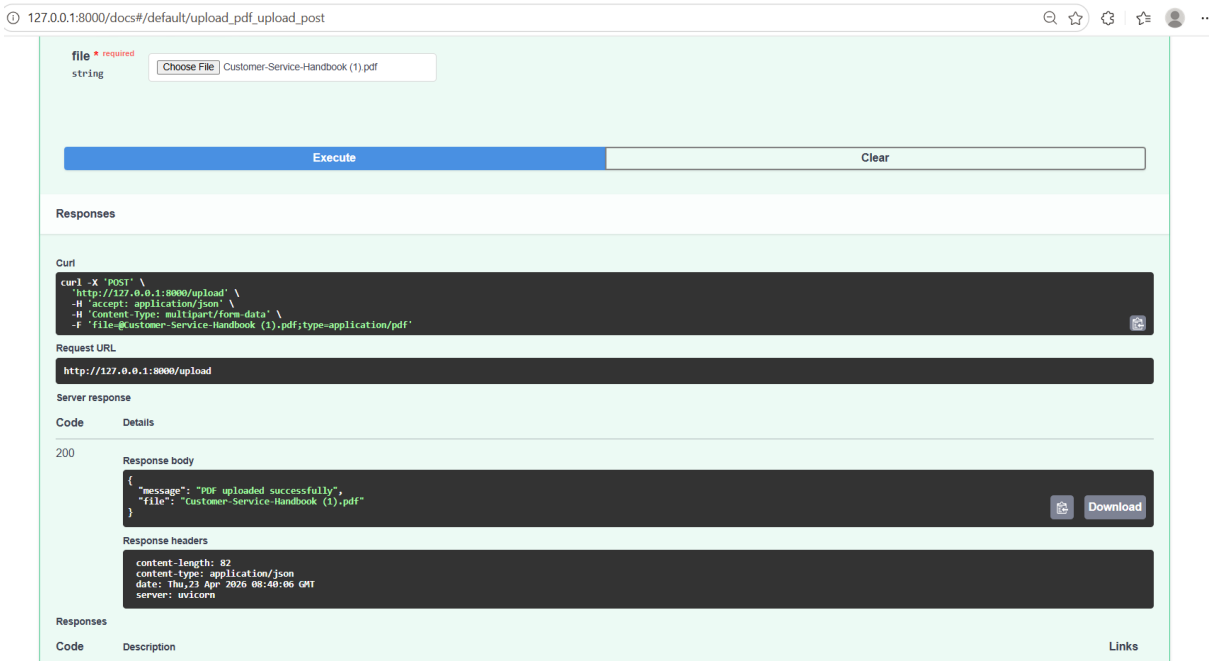
Test Home API



Test Health API



Upload PDF



Ask Question